Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik
Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

Lenguajes Regulares

Definiciones preliminares

Una cadena (o palabra) es una secuencia finita de símbolos.

Ejemplo: a, b, c son símbolos. $abcb$ es una cadena, de longitud 4.

ε denota la cadena vacía, de longitud 0: $|\varepsilon| = 0$

Un alfabeto es un conjunto finito de símbolos.

Un lenguaje es un conjunto de cadenas de símbolos.

Ejemplo: El conjunto de palíndromos sobre el alfabeto $\{0, 1\}$ es un lenguaje infinito: $\{\varepsilon, 0, 1, 00, 11, 010, 101, ...\}$.

Σ^* denota el lenguaje formado por todas las cadenas posibles sobre un alfabeto Σ.

Ejemplo: Si $\Sigma = \{a\}$, entonces $\Sigma^* = \{\varepsilon, a, aa, aaa, ...\}$

Ejemplo: Si $\Sigma = \{0, 1\}$, entonces $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, ...\}$

Definiciones preliminares

Sean Σ un alfabeto, L, L_1, L_2 lenguajes sobre Σ.

$L_1L_2 = \{xy \mid x \in L_1, y \in L_2\}$ (concatenación de L_1 y L_2)

$L^0 = \{\varepsilon\}$; $L^i = LL^{i-1}$(para $i \geq 1$)

Clausura (de Kleene): $L^* = \bigcup_{i=0}^{\infty} L^i$ incluye ε

Clausura positiva: $L^+ = \bigcup_{i=1}^{\infty} L^i$

Ejemplos:

- Si $L_1 = \{a, e\}$, $L_2 = \{x, y\}$, entonces $L_1L_2 = \{ax, ay, ex, ey\}$.
- Si $L_1 = \{a\}$, $L_2 = \emptyset$, entonces $L_1L_2 = \emptyset$.
- Si $L_1 = \{a\}$, $L_2 = \{\varepsilon\}$, entonces $L_1L_2 = \{a\}$.
- Si $L = \{a, b\}$, entonces $L^0 = \{\varepsilon\}$; $L^1 = L$; $L^2 = \{aa, ab, ba, bb\}$.

Expresiones regulares

Las expresiones regulares (ER) sobre un alfabeto Σ se definen de la siguiente manera:

1. $\varnothing$ es una ER y denota el lenguaje vacío.
2. ε es una ER y denota el lenguaje $\{\varepsilon\}$.
3. Para cada símbolo $a \in \Sigma$, a es una ER y denota el lenguaje $\{a\}$.
4. Si r y s son expresiones regulares que denotan los lenguajes R y S respectivamente, entonces:

   $r + s$ es una ER y denota $R \cup S$,
   rs es una ER y denota RS,
   r^* es una ER y denota R^*.

Ejemplos:

ER	Lenguaje
$(sa)^*$	$\{\varepsilon, sa, sasa, sasasa, ...\}$
$(s + a)^*$	$\{\varepsilon, s, a, ss, sa, as, aa, sss, ssa, ...\}$
$s(a^* + p)$	$\{sp, s, sa, saa, saaa, ...\}$

Ejercicio: Construir una ER sobre $\Sigma = \{0, 1\}$ de cadenas que terminen en 101 y otra de cadenas que tengan 00 o 11 al menos una vez.

Expresiones regulares en acción: **bash**

Ejemplo: Queremos leer un archivo **access.log** y listar las IPs ordenadas por frecuencia:

```
203.0.113.66 - - [07/Oct/2026:08:01:26 -0300] "GET /admin HTTP/1.1" 401 234 "-" "python-r
192.0.2.15 - - [07/Oct/2026:08:01:48 -0300] "GET /static/style.css HTTP/1.1" 200 1715 "-"
198.51.100.23 - - [07/Oct/2026:08:02:20 -0300] "GET /.git/config HTTP/1.1" 404 133 "-" "N
203.0.113.66 - - [07/Oct/2026:08:03:36 -0300] "GET /api/alumnos HTTP/1.1" 401 349 "-" "py
198.51.100.23 - - [07/Oct/2026:08:04:56 -0300] "GET /config.json HTTP/1.1" 404 123 "-" "N
192.0.2.31 - - [07/Oct/2026:08:05:21 -0300] "GET /api/alumnos HTTP/1.1" 200 5694 "-" "Moz
203.0.113.66 - - [07/Oct/2026:08:05:39 -0300] "POST /admin HTTP/1.1" 401 429 "-" "python-
192.0.2.39 - - [07/Oct/2026:08:06:17 -0300] "GET /contacto HTTP/1.1" 200 1471 "-" "Mozill
198.51.100.23 - - [07/Oct/2026:08:07:32 -0300] "GET /wp-admin HTTP/1.1" 404 436 "-" "Nikt
192.0.2.14 - - [07/Oct/2026:08:08:23 -0300] "GET / HTTP/1.1" 200 1487 "-" "Mozilla/5.0 (W
192.0.2.34 - - [07/Oct/2026:08:08:57 -0300] "GET /api/alumnos HTTP/1.1" 200 6157 "-" "Moz
192.0.2.23 - - [07/Oct/2026:08:09:22 -0300] "GET /api/alumnos HTTP/1.1" 200 2983 "-" "Moz
192.0.2.20 - - [07/Oct/2026:08:10:35 -0300] "GET /login HTTP/1.1" 200 3778 "-" "Mozilla/5
192.0.2.13 - - [07/Oct/2026:08:11:21 -0300] "GET /cursos/automatas HTTP/1.1" 200 6752 "-"
203.0.113.66 - - [07/Oct/2026:08:12:00 -0300] "GET /admin HTTP/1.1" 401 455 "-" "python-r
...
```

Comando bash:

```
grep -Eo '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' access.log | sort | uniq -c | sort -rn
```

Documentación: `https://tldp.org/LDP/Bash-Beginners-Guide/html/sect_04_01.html`

Expresiones regulares en acción: **Python**

Mismo ejemplo:

```
import re
from collections import Counter

patron:str = r'[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+'

ips:list[str] = []
with open('access.log') as f:
    for linea in f:
        ips.extend(re.findall(patron, linea))

for ip, n in Counter(ips).most_common():
    print(f'{n:7d} {ip}')
```

Documentación: `https://www.w3schools.com/python/python_regex.asp`

Expresiones regulares en acción: **SQL**

Mismo ejemplo, suponiendo que tenemos una tabla `logs` con una única columna `texto` con cada entrada cruda del log:

```
SELECT count(*) AS n, m[1] AS ip
FROM logs
CROSS JOIN LATERAL regexp_matches(texto,
                    '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+', 'g') AS m
GROUP BY m[1]
ORDER BY n DESC, ip;
```

Documentación: `https://www.geeksforgeeks.org/sql/regular-expressions-in-sql/`

Gramáticas regulares

Una gramática regular (GR) es una tupla (V, T, P, S):

- V es un conjunto de variables (no terminales)
- T es un conjunto de terminales
- P es un conjunto de producciones (reglas) de la forma:

$$A \to aB$$
$$A \to a$$
$$A \to \varepsilon \qquad \text{donde } A, B \in V;\ \ a \in T$$

- $S \in V$ es una variable especial llamada símbolo inicial.

Derivación: $\alpha \overset{*}{\underset{G}{\Rightarrow}} \beta$ ("α deriva β") si, comenzando en la cadena α, podemos aplicar 0 o más producciones de P para llegar a la cadena β.

Lenguaje generado por una gramática G: $L(G) = \left\{ w \in T^* \mid S \overset{*}{\underset{G}{\Rightarrow}} w \right\}$

Gramáticas regulares

Ejemplo: El lenguaje $0(10)^*$ es generado por esta gramática regular:

$$S \to 0A$$
$$A \to 1B$$
$$A \to \varepsilon$$
$$B \to 0A$$

Teorema: Toda gramática con producciones de la forma $A \to wB$ o $A \to w$, con $A, B \in V$, $w \in T^*$, puede reescribirse como una GR. (Demostración: ejercicio.)

Ejemplo: El lenguaje $0(10)^*$ es generado por la gramática: $S \to 0A, \;\; A \to 10A, \;\; A \to \varepsilon$.

Teorema: L es generado por una gramática regular si y sólo si L es un lenguaje regular.

Demo: Hopcroft & Ullman (1979), "Introduction to Automata Theory, Languages, and Computation", **1st Edition**, Addison-Wesley. Sección 9.1, teorema 9.1. Requiere leer primero sobre autómatas finitos.

Ejemplo: $L = \{a^n b^n \mid n \geq 1\}$ **no** es regular

Demostración: Supongamos que L es regular. Entonces existe una gramática regular G que lo genera, con producciones de la forma $A \to wB$, o $A \to w$, o $A \to \varepsilon$.

Sea $n = |V|$. Consideremos $w = a^n b^n \in L$. Como cada producción genera exactamente un terminal y deja a lo sumo una variable, la derivación de w es de la forma

$$S = A_0 \Rightarrow aA_1 \Rightarrow aaA_2 \Rightarrow \cdots \Rightarrow a^n A_n \Rightarrow \cdots \Rightarrow a^n b^n,$$

donde A_i es la variable que queda tras i pasos. La derivación tiene al menos n pasos, de modo que aparecen las variables $A_0, A_1, \ldots, A_n$. Son $n+1$ variables y G solo tiene n, luego se repite alguna: existen índices $0 \leq i < j \leq n$ tales que $A_i = A_j = A$:

$$S \Rightarrow^* a^i A \Rightarrow^* a^i a^{j-i} A \Rightarrow^* a^i a^{j-i} a^{n-j} b^n = a^n b^n$$

Podemos omitir la derivación $A \Rightarrow^* a^{j-i} A$, tomamos un "atajo" y obtenemos:

$$S \Rightarrow^* a^i A \Longrightarrow^* a^i a^{n-j} b^n = a^{n-j+i} b^n$$

Por lo tanto, $a^{n-j+i} b^n \in L$. Como $j - i > 0$, esta palabra tiene más b's que a's, y entonces **no pertenece** a L. Contradicción. Concluimos que L no es regular.